

RELAZIONE PROGETTO FINALE PYGAME - 3B TLC

LOPEZ - EPIFANI

Dati del progetto:

- Nome e cognome: Lopez Andrea e Epifani Massimo
- Classe: 3B TLC
- Nome del gioco: Monkey Market
- Data di consegna: 03/06/2026

Idea del gioco:

- Che tipo di gioco è?

Il nostro gioco è di tipo gestionale, in cui si può anche stare inattivi. Nel gioco si è una scimmia che vende prodotti per espandere il suo supermercato.

- Qual è l'obiettivo del giocatore?

L'obiettivo del giocatore è quello di vendere più prodotti per espandersi e vendere altri prodotti per guadagnare di più.

- Che cosa rende il tuo gioco un minimo "originale" o diverso da un esempio trovato online?

Il nostro gioco è diverso perché, a differenza di quello online, è una versione più semplice che offre meno servizi, ma rimane comunque divertente da giocare, anche se più veloce da completare.

- "In cosa consiste il gioco"?

Il gioco consiste nel vendere i prodotti per guadagnare soldi e comprare altri prodotti da vendere. Con i soldi guadagnati si possono anche comprare degli aggiornamenti per la scimmia controllata dal giocatore e per gli aiutanti per fargli portare più prodotti per velocizzare il processo di riempimento degli scaffali.

- "Chi è il pubblico a cui lo immagino destinato (bambini, compagni, appassionati di...)?"

Noi immaginiamo che il gioco sia destinato a dei ragazzi che si annoiano e cercano dei giochi gratis online per divertirsi o per passare il tempo.

Analisi e progettazione:

3.1 Regole del gioco

- Regole principali:

Il giocatore deve riempire gli scaffali con il cibo e poi spostarsi in cassa per venderli. Con i soldi guadagnati si sbloccano nuovi prodotti da vendere e si possono assumere degli aiutanti per velocizzare la vendita.

- Punteggi, vite, livelli, timer, ecc.:

Il gioco ha un solo livello e una volta completato va avanti all'infinito finché gli scaffali sono pieni.

3.2 Struttura del programma

Moduli e librerie utilizzate

- **Pygame**: Gestisce la finestra di gioco, la grafica (disegna personaggi e prodotti), i comandi della tastiera/mouse e la fluidità dei fotogrammi (FPS).
- **Math**: Calcola le distanze geometriche per far muovere i clienti verso gli scaffali e gestisce le oscillazioni per le animazioni di camminata.
- **Random**: Genera scelte casuali, come il tipo di prodotto richiesto dai clienti, la quantità della loro spesa e da dove farli entrare nel negozio.
- **Sys**: Si occupa di chiudere correttamente e in modo sicuro il programma e la finestra di gioco quando clicchi sulla "X".

Le parti principali del programma

1. Gestione del personaggio (Il Giocatore)

- **Movimento**: Modifica continuamente le coordinate X e Y della scimmietta in base ai tasti premuti (frece o WASD).
- **Stato e Inventario**: Memorizza in una lista dinamica quanti e quali prodotti (banane, uova, grano) la scimmia ha in mano in ogni momento.
- **Limiti (Collisioni)**: Blocca il passaggio del personaggio se tenta di scavalcare gli scaffali o di uscire dai bordi della mappa.

2. Gestione dei nemici / ostacoli (Clienti e Imprevisti)

- **Intelligenza Artificiale (AI) dei Clienti:** Sposta i clienti facendoli camminare in sequenza dall'ingresso allo scaffale, poi alla cassa e infine all'uscita.
- **Ostacoli e Imprevisti:** Controlla i timer che bloccano gli aiutanti quando si addormentano o che attivano i ladri, costringendo il giocatore a intervenire.

3. Gestione del punteggio (Il Portafoglio)

- **Variabile del Denaro:** Gestisce il portafoglio del giocatore tramite un contatore numerico che aumenta a ogni pagamento.
- **Transazioni:** Verifica se il denaro posseduto è sufficiente per sbloccare nuovi banchi, sottraendo il costo dal totale in caso di acquisto.
- **Interfaccia Grafica (UI):** Trasforma il valore numerico dei soldi in testo colorato impresso sullo schermo.

4. Schermata iniziale / Menu e Stati del gioco

- **Menu Principale:** Blocca il gioco all'avvio mostrando una copertina statica con il titolo e un pulsante per iniziare.
- **Cambio di Stato:** Switcha il programma da una modalità all'altra (es. da "MENU" a "GIOCO") non appena rileva il click del mouse sul pulsante.
- **Schermate Secondarie:** Congela l'azione di gioco e mostra i relativi pannelli grafici quando l'utente attiva la pausa o subisce un Game Over.

3.3 Strutture dati importanti

Classi:

- **Player:** La tua scimmietta (il protagonista) con i suoi movimenti e lo zaino.
- **Structure:** I banchi di raccolta, gli scaffali di vendita, la gallina, la cassa e il cestino.
- **Helper:** Gli assistenti automatizzati che lavorano al posto tuo.
- **Customer:** I clienti che entrano nel negozio con i loro desideri di spesa.
- **DeliverooRider:** Il fattorino speciale che arriva per ordinare grandi quantità di merce.
- **DepartmentBanner:** I cartelloni in legno che indicano il nome del reparto (Banane, Grano, Uova).
- **UpgradeButton:** I tre pulsanti cliccabili a destra per potenziare zaino, velocità e dipendenti.

Liste:

- **self.items**: Lo zaino dei personaggi (contiene i testi dei frutti trasportati, es. ['banana', 'egg']).
- **structures**: L'arredamento totale del negozio inserito nella mappa.
- **helpers_list**: I dipendenti attualmente assunti e attivi.
- **customers**: Tutti i clienti che stanno camminando nel supermercato in quel momento.
- **cash_queue**: La fila ordinata dei clienti davanti alla cassa.
- **unlocked_product_types**: La lista delle merci che il negozio può vendere in base a cosa hai sbloccato.
- **upgrades**: I pulsanti di potenziamento visualizzati nel menu laterale.

Tuple:

- **I Colori** (es. (100, 190, 100)): I codici numerici RGB per colorare lo sfondo, i personaggi e i prodotti.
- **Le Misure** (es. (1000, 700)): La larghezza e l'altezza fisse in pixel della finestra di gioco.

4. Sviluppo e uso dell'AI

4.1 Come hai lavorato (tu/gruppo)

- **Chi ha fatto cosa:**

Lopez: ha prevalentemente lavorato con l'intelligenza artificiale generando il codice e modificandolo.

Epifani: ha controllato il gioco provandolo e comunicando gli errori o eventuali mancanze nel gioco. La relazione l'abbiamo divisa più o meno a metà.

- **Come vi siete divisi i compiti:**

Lopez: Codice e relazione.

Epifani: Test del gioco e relazione.

- **Eventuali problemi di comunicazione/organizzazione e come li avete risolti:**

Questi problemi li abbiamo risolti attraverso chiamate o videochiamate il pomeriggio quando entrambi eravamo liberi.

4.2 Come hai usato l'AI (obbligatorio se l'hai usata)

- **Quali strumenti hai usato:**

Come intelligenza artificiale abbiamo usato Gemini AI.

- **Per quali parti del progetto li hai usati:**

Abbiamo utilizzato l'intelligenza artificiale per generare e modificare il codice e per correggere eventuali bug durante la creazione del gioco.

- **Alcuni prompt principali che hai usato, ad esempio:**

Fai sì che all'inizio puoi portare solo 3 banane e per portarne di più bisogna aggiornare il personaggio con i soldi guadagnati. Fai sì che gli omini siano più realistici ma in 2D e che ci siano più cose da vendere: ad esempio pagando si sbloccano le pannocchie e pagando si sblocca un'altra pianta di banane o si compra un'altra pianta di pannocchie.

Fai sì che le scimmie siano sempre a testa in su, la capacità massima è 10. Aggiungi un cestino per buttare le cose. Fai arrivare un deliveroo ogni 3 minuti di gioco che ti chiede un tot di oggetti da vendere in cambio di soldi.

5. Test e risultati

5.1 Casi di test

Test	Cosa ho fatto	Cosa mi aspettavo	Cosa è successo
1 (Risolto)	Raccolgo un uovo dalla postazione della gallina.	L'uovo si aggiunge allo zaino della scimmia senza problemi.	CRASH DEL GIOCO: Il programma si bloccava all'istante a causa di un loop infinito nel calcolo del trasferimento (Risolto inserendo il timer di <code>transfer_cooldown</code>).
2 (Risolto)	Sblocco il nuovo aiutante nel reparto uova.	La scimmietta dipendente appare e inizia a fare la spola tra gallina e scaffali.	BUG DEL PATHFINDING: L'aiutante non trovava la sorgente e camminava all'infinito verso un angolo vuoto (Risolto mappando la gallina come <code>processor</code> attivo).

5.2 Bug e problemi

Ovviamente ci sono stati anche dei bug e dei problemi durante la creazione del gioco:

- Un bug, ad esempio, è stato quando provavo a recuperare le uova della gallina: davo il grano alla gallina, le lo “mangiava” e poi faceva le uova, tutto funzionava alla perfezione, fino a quando non provavo a recuperare le uova. Mi avvicinavo alla gallina, prendevo l'uovo, ma il gioco si bloccava completamente e non ripartiva.
- Poi volevamo fare anche un altro livello, ma non abbiamo avuto abbastanza tempo, sia per la data di consegna un po' ravvicinata (Prof per questa cosa non si arrabbi, per favore), sia perché non è stato molto il tempo in cui eravamo liberi entrambi e quindi ci siamo accontentati di un livello solo.

6. Conclusioni e riflessione personale

- **Che cosa hai imparato di nuovo (su Python, sui videogiochi, sul lavoro di gruppo, sull'uso dell'AI...)?**
- Durante questa esercitazione abbiamo imparato molte cose: quella più importante secondo noi, è il fatto che con l'AI bisogna avere una comunicazione chiara e approfondita.

- **Qual è stata la cosa più difficile?**
- La cosa più difficile, per l'appunto, è stata la comunicazione con l'AI, trovare i termini adatti affinché l'AI capisca perfettamente cosa le stiamo chiedendo e come deve risolvere il problema.

- **C'è stata una difficoltà che sei riuscito a superare? Come?**
- La difficoltà della comunicazione con l'AI siamo riusciti a risolverla pensando ai termini adatti al contesto e, ci siamo fatti anche aiutare cercando su google che termini potessimo utilizzare.

- **Se rifacessi il progetto da capo, che cosa faresti in modo diverso?**
- Se dovessi rifare il progetto da capo, probabilmente, cambierei l'approccio con l'AI: mi sono reso conto che non ho iniziato al meglio la mia richiesta della creazione del gioco. Avrei potuto essere un po' più specifico inizialmente, così da trovarmi poi avvantaggiato andando avanti con le richieste.

- **Pensi di aver usato l'AI in modo intelligente o ti sei appoggiato troppo?**
- A parere mio sono riuscito a utilizzare l'AI in modo intelligente, analizzando il problema del gioco e chiedendo all'AI di risolverlo, anche usando l'errore che mi dava python per far capire al 100% all'AI quale fosse il problema e dove si trovasse.

7. Allegati

Screenshots:

Inizio del gioco:



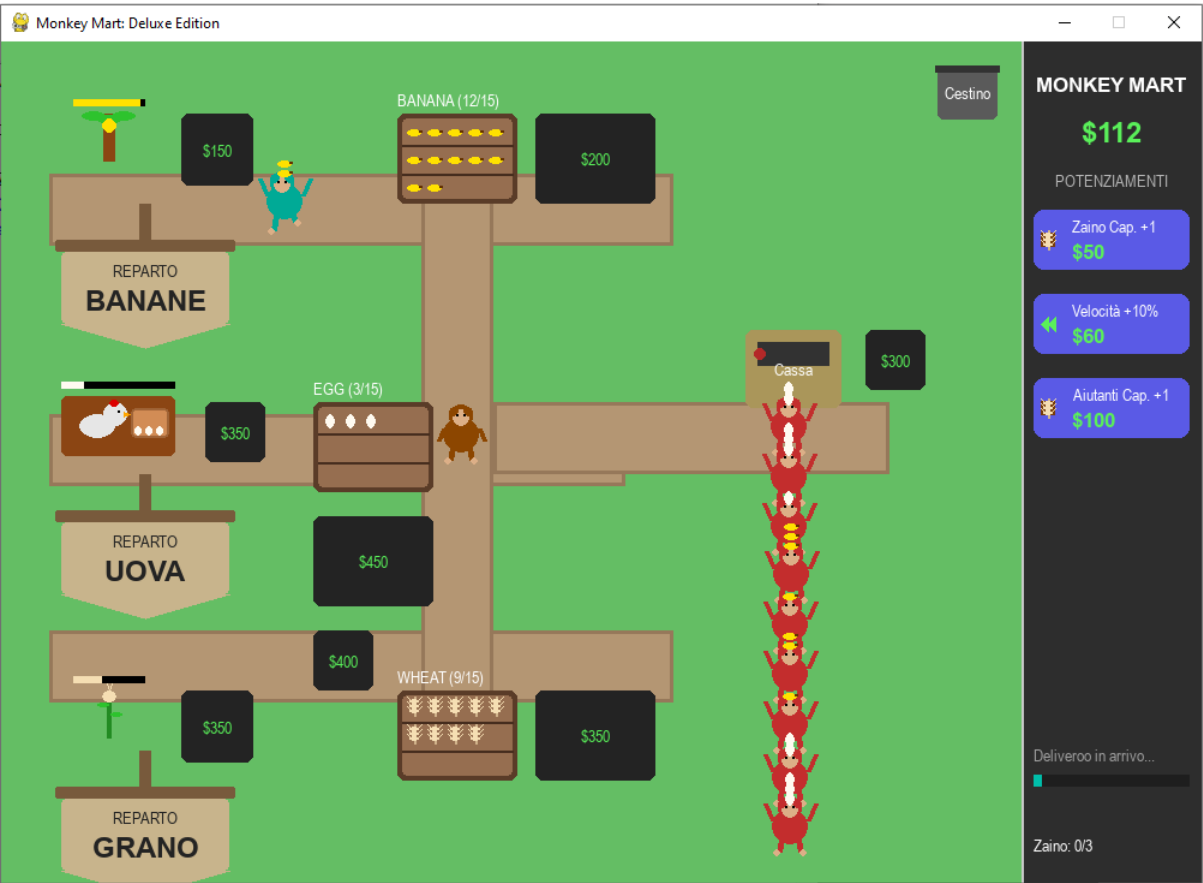
Con il deliveroo:



Con tutti i prodotti da vendere sbloccati:



Con un aiutante assunto:



Codice:

```

import pygame
import sys
import math
import random

# Inizializzazione di Pygame
pygame.init()

# --- COSTANTI DI GIOCO ---
SCREEN_WIDTH = 1000
SCREEN_HEIGHT = 700
UI_PANEL_WIDTH = 150
GAME_AREA_WIDTH = SCREEN_WIDTH - UI_PANEL_WIDTH
FPS = 60

# --- COLORI ---
COL_BG = (100, 190, 100)
COL_PATH = (180, 150, 115)
COL_PATH_BORDER = (150, 120, 90)
COL_UI_BG = (45, 45, 45)
COL_MONKEY_BODY = (140, 70, 0)
COL_MONKEY_FACE = (220, 175, 135)
COL_CUSTOMER = (195, 45, 45)
COL_DELIVEROO = (0, 190, 170)
COL_HELPER = (0, 170, 150)
COL_CASHIER = (210, 105, 30)
COL_PLANT_STEM = (34, 139, 34)
COL_LEAF = (45, 195, 45)
COL_SHELF = (150, 110, 80)
COL_CASH = (170, 150, 90)
COL_TRASH = (90, 90, 90)
COL_BANANA = (255, 225, 0)

# REPARTO GRANO E UOVA
COL_WHEAT = (245, 222, 179)
COL_WHEAT_STEM = (210, 180, 140)
COL_EGG = (255, 250, 240)
COL_TEXT = (255, 255, 255)
COL_MONEY = (90, 240, 90)
COL_LOCK = (35, 35, 35)
COL_BTN = (90, 90, 230)
COL_BANNER = (120, 90, 60)
COL_BANNER_PENNANT = (200, 180, 140)

# Stato Globale Upgrades
GLOBAL_HELPER_CAP = 2

# Configurazione Finestra

```

```

screen = pygame.display.set_mode((SCREEN_WIDTH, SCREEN_HEIGHT))
pygame.display.set_caption("Monkey Mart: Deluxe Edition")
clock = pygame.time.Clock()

font_sm = pygame.font.SysFont("Arial", 14)
font_md = pygame.font.SysFont("Arial", 17, bold=True)
font_lg = pygame.font.SysFont("Arial", 24, bold=True)

# --- FUNZIONI DI DISEGNO GRAFICA 2D ---
def draw_monkey(surface, x, y, color, is_carrying=False, walk_tick=0, is_moving=False,
scarf_color=None):
    s = pygame.Surface((50, 60), pygame.SRCALPHA)
    left_leg_offset = 0
    right_leg_offset = 0
    head_sway = 0

    if is_moving:
        left_leg_offset = math.sin(walk_tick * 0.2) * 5
        right_leg_offset = -math.sin(walk_tick * 0.2) * 5
        head_sway = math.cos(walk_tick * 0.15) * 2

    pygame.draw.line(s, color, (18, 36), (16, int(46 + left_leg_offset)), 5)
    pygame.draw.line(s, color, (32, 36), (34, int(46 + right_leg_offset)), 5)
    pygame.draw.circle(s, COL_MONKEY_FACE, (14, int(47 + left_leg_offset)), 3)
    pygame.draw.circle(s, COL_MONKEY_FACE, (36, int(47 + right_leg_offset)), 3)

    pygame.draw.ellipse(s, color, (10, 10 + int(abs(head_sway)/2), 30, 30))
    head_x = int(25 + head_sway)
    pygame.draw.circle(s, color, (head_x, 10), 10)
    pygame.draw.circle(s, COL_MONKEY_FACE, (head_x, 11), 7)

    pygame.draw.circle(s, color, (head_x - 6, 6), 4)
    pygame.draw.circle(s, color, (head_x + 6, 6), 4)
    pygame.draw.circle(s, (0,0,0), (head_x - 3, 9), 1)
    pygame.draw.circle(s, (0,0,0), (head_x + 3, 9), 1)

    if scarf_color:
        pygame.draw.rect(s, scarf_color, (head_x - 8, 18, 16, 6), border_radius=2)
        pygame.draw.rect(s, scarf_color, (head_x - 3, 22, 6, 10), border_radius=1)

    if is_carrying:
        pygame.draw.line(s, color, (10, 20), (4, 6), 4)
        pygame.draw.line(s, color, (40, 20), (46, 6), 4)
    else:
        pygame.draw.line(s, color, (10, 20), (6, 30), 4)
        pygame.draw.line(s, color, (40, 20), (44, 30), 4)

    rect = s.get_rect(center=(int(x), int(y)))

```

```

surface.blit(s, rect)

def draw_banana_item(surface, x, y):
    pygame.draw.ellipse(surface, COL_BANANA, (x-6, y-3, 12, 6))
    pygame.draw.rect(surface, (100,50,0), (x+4, y-2, 3, 2))

def draw_wheat_item(surface, x, y):
    pygame.draw.line(surface, COL_WHEAT_STEM, (x, y-8), (x, y+8), 2)
    for i in range(3):
        offset_y = -6 + (i * 4)
        pygame.draw.ellipse(surface, COL_WHEAT, (x-4, y + offset_y, 8, 4))
        pygame.draw.line(surface, COL_WHEAT, (x-4, y+offset_y), (x-6, y+offset_y-2), 1)
        pygame.draw.line(surface, COL_WHEAT, (x+4, y+offset_y), (x+6, y+offset_y-2), 1)

def draw_egg_item(surface, x, y):
    pygame.draw.ellipse(surface, COL_EGG, (x-4, y-6, 8, 12))

def draw_banana_plant(surface, x, y):
    pygame.draw.rect(surface, (139,69,19), (x-5, y, 10, 40))
    for i in range(3):
        angle = i * 60 + 30
        lx = x + math.cos(math.radians(angle)) * 15
        ly = y + math.sin(math.radians(angle)) * 5
        pygame.draw.ellipse(surface, COL_LEAF, (lx-10, ly-4, 20, 8))
    pygame.draw.circle(surface, COL_BANANA, (x, y+10), 6)

def draw_wheat_plant(surface, x, y):
    pygame.draw.rect(surface, COL_PLANT_STEM, (x-2, y, 4, 40))
    pygame.draw.ellipse(surface, COL_WHEAT, (x-6, y, 12, 10))
    pygame.draw.line(surface, COL_WHEAT, (x, y), (x-3, y-8), 1)
    pygame.draw.line(surface, COL_WHEAT, (x, y), (x+3, y-8), 1)
    pygame.draw.ellipse(surface, COL_LEAF, (x-10, y+10, 10, 4))
    pygame.draw.ellipse(surface, COL_LEAF, (x+1, y+18, 10, 4))

def draw_hen(surface, x, y):
    pygame.draw.ellipse(surface, (230, 230, 230), (x-15, y-10, 30, 20))
    pygame.draw.circle(surface, (230, 230, 230), (x+15, y-10), 10)
    pygame.draw.polygon(surface, (255, 165, 0), [(x+22, y-12), (x+28, y-10), (x+22, y-8)])
    pygame.draw.circle(surface, (0,0,0), (x+18, y-12), 1)
    pygame.draw.ellipse(surface, (220, 0, 0), (x+10, y-22, 8, 6))

# --- CLASSI DI LOGICA ---
class DepartmentBanner:
    def __init__(self, x, y, text):
        self.x = x
        self.y = y
        self.text = text
        self.rect = pygame.Rect(x, y, 140, 60)

```

```

def draw(self, surface):
    pygame.draw.rect(surface, COL_BANNER, (self.x - 5, self.y - 10, 150, 10),
border_radius=2)
    pygame.draw.rect(surface, COL_BANNER, (self.x + 65, self.y - 40, 10, 40))

    pygame.draw.rect(surface, COL_BANNER_PENNANT, self.rect, border_radius=4)
    pygame.draw.polygon(surface, COL_BANNER_PENNANT, [
        (self.x, self.y + 60), (self.x + 140, self.y + 60),
        (self.x + 70, self.y + 80)
    ])

    txt = font_sm.render("REPARTO", True, COL_LOCK)
    txt_type = font_lg.render(self.text, True, COL_LOCK)
    surface.blit(txt, txt.get_rect(center=(self.rect.centerx, self.y + 15)))
    surface.blit(txt_type, txt_type.get_rect(center=(self.rect.centerx, self.y + 40)))

```

```

class Player:

```

```

    def __init__(self):
        self.x = 450
        self.y = 330
        self.radius = 18
        self.inventory_max = 3
        self.speed = 2.4
        self.items = []
        self.money = 0
        self.walk_tick = 0
        self.is_moving = False
        self.transfer_cooldown = 0
        self.current_unlocking_struct = None
        self.unlock_timer = 0

```

```

    def move(self, keys):

```

```

        dx, dy = 0, 0
        if keys[pygame.K_LEFT] or keys[pygame.K_a]: dx = -1
        if keys[pygame.K_RIGHT] or keys[pygame.K_d]: dx = 1
        if keys[pygame.K_UP] or keys[pygame.K_w]: dy = -1
        if keys[pygame.K_DOWN] or keys[pygame.K_s]: dy = 1

```

```

        if dx != 0 or dy != 0:

```

```

            self.is_moving = True
            self.walk_tick += 1
            dist = math.hypot(dx, dy)
            self.x += (dx / dist) * self.speed
            self.y += (dy / dist) * self.speed

```

```

        else:

```

```

            self.is_moving = False

```

```

self.x = max(self.radius, min(GAME_AREA_WIDTH - self.radius, self.x))
self.y = max(self.radius, min(SCREEN_HEIGHT - self.radius, self.y))

def draw(self, surface):
    is_carrying = len(self.items) > 0
    draw_monkey(surface, self.x, self.y, COL_MONKEY_BODY, is_carrying, self.walk_tick,
self.is_moving)
    for i, item_type in enumerate(self.items):
        iy = self.y - 28 - (i * 8)
        if item_type == 'banana': draw_banana_item(surface, self.x, iy)
        elif item_type == 'wheat': draw_wheat_item(surface, self.x, iy)
        elif item_type == 'egg': draw_egg_item(surface, self.x, iy)

class Helper:
    def __init__(self, x, y, product_type):
        self.x = x
        self.y = y
        self.product_type = product_type
        self.radius = 16
        self.speed = 0.95
        self.items = []
        self.inventory_max = GLOBAL_HELPER_CAP
        self.walk_tick = 0
        self.is_moving = False
        self.transfer_cooldown = 0
        self.state = "VAI_A_POSTAZIONE" if product_type == "cashier" else "VAI_A_FONTE"

    def update(self, sources, shelves, cash_desk, hen_station):
        self.inventory_max = GLOBAL_HELPER_CAP
        if self.transfer_cooldown > 0:
            self.transfer_cooldown -= 1

        if self.product_type == "cashier":
            tx, ty = cash_desk.rect.centerx, cash_desk.rect.top - 15
            dist = math.hypot(self.x - tx, self.y - ty)
            if dist > 4:
                self.is_moving = True
                self.walk_tick += 1
                dx, dy = tx - self.x, ty - self.y
                self.x += (dx / dist) * self.speed
                self.y += (dy / dist) * self.speed
            else:
                self.is_moving = False
                cash_desk.cashier_present = True
            return

        target_source = next((s for s in sources if s.active and s.product_type ==
self.product_type), None)

```



```
valid_shelves = [s for s in shelves if s.active and s.product_type == self.product_type]
target_shelf = min(valid_shelves, key=lambda s: s.stock) if valid_shelves else None
```

```
if self.product_type == 'egg' and not target_source and hen_station.active:
    target_source = hen_station
```

```
if not target_source or not target_shelf:
    self.is_moving = False
    return
```

```
if self.state == "VAI_A_FONTE":
    self.is_moving = True
    self.walk_tick += 1
    ty_offset = 10 if self.product_type != 'egg' else 45
    tx, ty = target_source.rect.centerx, target_source.rect.bottom + ty_offset
    self.move_towards(tx, ty)
    if math.hypot(self.x - tx, self.y - ty) < 5:
        if target_source.stock > 0:
            self.state = "RACCOGLI"
        else:
            self.is_moving = False
```

```
elif self.state == "RACCOGLI":
    self.is_moving = False
    if target_source.stock > 0 and len(self.items) < self.inventory_max:
        if self.transfer_cooldown == 0:
            target_source.stock -= 1
            self.items.append(self.product_type)
            self.transfer_cooldown = 12
        else:
            self.state = "VAI_A_SCAFFALE" if len(self.items) > 0 else "VAI_A_FONTE"
```

```
elif self.state == "VAI_A_SCAFFALE":
    self.is_moving = True
    self.walk_tick += 1
    tx, ty = target_shelf.rect.centerx, target_shelf.rect.bottom + 10
    self.move_towards(tx, ty)
    if math.hypot(self.x - tx, self.y - ty) < 5:
        self.state = "DEPOSITA"
```

```
elif self.state == "DEPOSITA":
    self.is_moving = False
    if target_shelf.stock < target_shelf.max_stock and len(self.items) > 0:
        if self.transfer_cooldown == 0:
            target_shelf.stock += 1
            self.items.pop()
            self.transfer_cooldown = 10
        else:
```

```

        self.state = "VAI_A_FONTE"

def move_towards(self, tx, ty):
    dx, dy = tx - self.x, ty - self.y
    dist = math.hypot(dx, dy)
    if dist > 0:
        self.x += (dx / dist) * self.speed
        self.y += (dy / dist) * self.speed

def draw(self, surface):
    is_carrying = len(self.items) > 0
    color = COL_CASHIER if self.product_type == "cashier" else COL_HELPER
    draw_monkey(surface, self.x, self.y, color, is_carrying, self.walk_tick, self.is_moving)
    for i, item_type in enumerate(self.items):
        iy = self.y - 28 - (i * 8)
        if item_type == 'banana': draw_banana_item(surface, self.x, iy)
        elif item_type == 'wheat': draw_wheat_item(surface, self.x, iy)
        elif item_type == 'egg': draw_egg_item(surface, self.x, iy)

class Structure:
    def __init__(self, x, y, w, h, col, type_str, prod_type=None, cost=0):
        self.rect = pygame.Rect(x, y, w, h)
        self.color = col
        self.type = type_str
        self.product_type = prod_type
        self.stock = 0
        self.input_stock = 0
        self.input_max_stock = 10
        self.max_stock = 15
        self.money_pool = 0
        self.active = (cost == 0)
        self.prod_timer = 0
        self.prod_speed = 340
        self.hen_speed = 350
        self.unlock_cost = cost
        self.player_present = False
        self.cashier_present = False

    def update(self):
        if not self.active: return
        if self.type == 'source':
            self.prod_timer += 1
            if self.prod_timer >= self.prod_speed:
                if self.stock < self.max_stock:
                    self.stock += 1
                self.prod_timer = 0
        elif self.type == 'processor':
            self.prod_timer += 1

```

```

if self.prod_timer >= self.hen_speed:
    if self.input_stock > 0 and self.stock < self.max_stock:
        self.input_stock -= 1
        self.stock += 1
        self.prod_timer = 0

def draw(self, surface, player_obj):
    if not self.active:
        pygame.draw.rect(surface, COL_LOCK, self.rect, border_radius=6)
        txt_cost = font_sm.render(f"${self.unlock_cost}", True, COL_MONEY)
        surface.blit(txt_cost, txt_cost.get_rect(center=self.rect.center))

    if player_obj.current_unlocking_struct == self:
        pct = player_obj.unlock_timer / 180
        bar_w = int(self.rect.width * pct)
        pygame.draw.rect(surface, (30, 30, 30), (self.rect.x, self.rect.bottom + 5,
self.rect.width, 6), border_radius=2)
        pygame.draw.rect(surface, (0, 255, 0), (self.rect.x, self.rect.bottom + 5, bar_w, 6),
border_radius=2)
        return

    if self.type == 'source':
        if self.product_type == 'banana': draw_banana_plant(surface, self.rect.centerx,
self.rect.y)
        elif self.product_type == 'wheat': draw_wheat_plant(surface, self.rect.centerx,
self.rect.y)

        fill_w = (self.stock / self.max_stock) * self.rect.width
        pygame.draw.rect(surface, (0,0,0), (self.rect.x, self.rect.y - 12, self.rect.width, 6))
        draw_col = COL_BANANA if self.product_type == 'banana' else COL_WHEAT
        pygame.draw.rect(surface, draw_col, (self.rect.x, self.rect.y - 12, fill_w, 6))

    elif self.type == 'processor':
        pygame.draw.rect(surface, (139, 69, 19), self.rect, border_radius=5)
        draw_hen(surface, self.rect.x + 30, self.rect.centery)

        basket_rect = pygame.Rect(self.rect.x + 58, self.rect.y + 10, 32, 25)
        pygame.draw.rect(surface, (210, 140, 85), basket_rect, border_radius=4)
        pygame.draw.rect(surface, (160, 100, 55), basket_rect, width=2, border_radius=4)

    for i in range(self.stock):
        row = i // 3
        col = i % 3
        ex = basket_rect.x + 6 + (col * 9)
        ey = basket_rect.bottom - 6 - (row * 6)
        if basket_rect.collidepoint(ex, ey):
            pygame.draw.ellipse(surface, COL_EGG, (ex-3, ey-4, 6, 8))

```

```

fill_w = (self.stock / self.max_stock) * self.rect.width
pygame.draw.rect(surface, (0,0,0), (self.rect.x, self.rect.y - 12, self.rect.width, 6))
pygame.draw.rect(surface, COL_EGG, (self.rect.x, self.rect.y - 12, fill_w, 6))

for i in range(self.input_stock):
    row = i // 5
    col = i % 5
    draw_wheat_item(surface, self.rect.x + 8 + col * 8, self.rect.bottom - 12 + row * 8)

elif self.type == 'shelf':
    pygame.draw.rect(surface, (90, 60, 40), self.rect, border_radius=6)
    inner = pygame.Rect(self.rect.x + 4, self.rect.y + 4, self.rect.width - 8, self.rect.height
- 8)
    pygame.draw.rect(surface, COL_SHELF, inner, border_radius=4)

    pygame.draw.line(surface, (70, 45, 30), (self.rect.x + 4, self.rect.y + 26),
(self.rect.right - 4, self.rect.y + 26), 2)
    pygame.draw.line(surface, (70, 45, 30), (self.rect.x + 4, self.rect.y + 50),
(self.rect.right - 4, self.rect.y + 50), 2)

    for i in range(self.stock):
        row = i // 5
        col = i % 5
        ix = self.rect.x + 14 + (col * 17)
        iy = self.rect.y + 16 + (row * 23)
        if self.product_type == 'banana': draw_banana_item(surface, ix, iy)
        elif self.product_type == 'wheat': draw_wheat_item(surface, ix, iy - 3)
        elif self.product_type == 'egg': draw_egg_item(surface, ix, iy)

    txt = font_sm.render(f"{self.product_type.upper()} ({self.stock}/15)", True,
COL_TEXT)
    surface.blit(txt, (self.rect.x, self.rect.y - 20))

elif self.type == 'cash':
    pygame.draw.rect(surface, self.color, self.rect, border_radius=6)
    pygame.draw.rect(surface, (50,50,50), (self.rect.x+10, self.rect.y+10, self.rect.width-
20, 20))

    light_col = COL_MONEY if (self.player_present or self.cashier_present) else
(180,40,40)
    pygame.draw.circle(surface, light_col, (self.rect.x + 12, self.rect.y + 20), 5)

    if self.money_pool > 0:
        txt_m = font_md.render(f"${self.money_pool}", True, COL_MONEY)
        surface.blit(txt_m, txt_m.get_rect(center=self.rect.center))
    else:
        txt_c = font_sm.render("Cassa", True, COL_TEXT)
        surface.blit(txt_c, txt_c.get_rect(center=self.rect.center))

```

```

elif self.type == 'trash':
    pygame.draw.rect(surface, COL_TRASH, self.rect, border_radius=5)
    pygame.draw.rect(surface, (50,50,50), (self.rect.x-2, self.rect.y, self.rect.width+4, 6))
    txt_t = font_sm.render("Cestino", True, COL_TEXT)
    surface.blit(txt_t, txt_t.get_rect(center=self.rect.center))

class Customer:
    def __init__(self, unlocked_products):
        self.x = -20
        self.y = random.choice([140, 360, 520])
        self.speed = 1.1
        self.wants_product = random.choice(unlocked_products) if len(unlocked_products) > 0
    else 'banana'
        self.state = "VAI_A_SCAFFALE"
        self.timer = 0
        self.items = []
        self.max_items = random.randint(1, 3)
        self.walk_tick = 0
        self.is_moving = False
        self.assigned_shelf = None

    def update(self, shelves, cash_desk, cash_queue):
        valid_shelves = [s for s in shelves if s.active and s.product_type == self.wants_product]

        if not valid_shelves and self.state == "VAI_A_SCAFFALE":
            self.state = "ESCI"

        if self.state == "VAI_A_SCAFFALE":
            if not self.assigned_shelf or not self.assigned_shelf.active or
            (self.assigned_shelf.stock == 0 and any(s.stock > 0 for s in valid_shelves)):
                shelves_with_stock = [s for s in valid_shelves if s.stock > 0]
                self.assigned_shelf = random.choice(shelves_with_stock) if shelves_with_stock
            else valid_shelves[0]

            self.is_moving = True
            self.walk_tick += 1
            tx, ty = self.assigned_shelf.rect.centerx, self.assigned_shelf.rect.bottom + 15
            self.move_towards(tx, ty)
            if math.hypot(self.x - tx, self.y - ty) < 4:
                self.state = "PRENDI"
                self.timer = 0

        elif self.state == "PRENDI":
            self.is_moving = False
            if self.assigned_shelf and self.assigned_shelf.stock > 0 and len(self.items) <
            self.max_items:
                self.timer += 1

```

```

    if self.timer >= 25:
        self.assigned_shelf.stock -= 1
        self.items.append(self.wants_product)
        self.timer = 0
    else:
        if len(self.items) > 0:
            self.state = "VAI_A_CASSA"
        else:
            self.state = "ESCI"

elif self.state in ("VAI_A_CASSA", "PAGA"):
    idx = cash_queue.index(self) if self in cash_queue else 0
    tx = cash_desk.rect.centerx
    ty = cash_desk.rect.bottom + 20 + (idx * 42)

    dist = math.hypot(self.x - tx, self.y - ty)
    if dist > 4:
        self.is_moving = True
        self.walk_tick += 1
        self.move_towards(tx, ty)
    else:
        self.is_moving = False
        if idx == 0:
            self.state = "PAGA"
        else:
            self.state = "VAI_A_CASSA"

if self.state == "PAGA":
    if not cash_desk.player_present and not cash_desk.cashier_present:
        return False

    if self.timer == 0:
        self.timer = 45 * len(self.items)

    self.timer -= 1
    if self.timer <= 0:
        for item in self.items:
            # RISOLTO: Nuovi bilanciamenti dei prezzi richiesti dall'utente
            if item == 'banana': pay_amount = 8
            elif item == 'wheat': pay_amount = 10
            elif item == 'egg': pay_amount = 12
            cash_desk.money_pool += pay_amount
        self.items = []
        self.state = "ESCI"

elif self.state == "ESCI":
    self.is_moving = True
    self.walk_tick += 1

```

```

        self.move_towards(-40, self.y)
        if self.x < -30: return True
    return False

def move_towards(self, tx, ty):
    dx, dy = tx - self.x, ty - self.y
    dist = math.hypot(dx, dy)
    if dist > 0:
        self.x += (dx / dist) * self.speed
        self.y += (dy / dist) * self.speed

def draw(self, surface):
    draw_monkey(surface, self.x, self.y, COL_CUSTOMER, len(self.items) > 0,
self.walk_tick, self.is_moving)
    for i, item_type in enumerate(self.items):
        iy = self.y - 28 - (i * 8)
        if item_type == 'banana': draw_banana_item(surface, self.x, iy)
        elif item_type == 'wheat': draw_wheat_item(surface, self.x, iy)
        elif item_type == 'egg': draw_egg_item(surface, self.x, iy)

class DeliverooRider:
    def __init__(self, unlocked_products):
        self.x = 550
        self.y = SCREEN_HEIGHT + 40
        self.speed = 1.6
        self.product_type = random.choice(unlocked_products) if len(unlocked_products) > 0
    else 'banana'
        self.qty_needed = random.randint(3, 6)
        self.qty_delivered = 0
        self.reward = self.qty_needed * 30
        self.state = "ENTRA"
        self.active = True
        self.walk_tick = 0
        self.is_moving = False

def update(self, player_rect, player):
    if self.state == "ENTRA":
        self.is_moving = True
        self.walk_tick += 1
        tx, ty = 600, 420
        dx, dy = tx - self.x, ty - self.y
        dist = math.hypot(dx, dy)
        if dist > 4:
            self.x += (dx / dist) * self.speed
            self.y += (dy / dist) * self.speed
        else: self.state = "ASPETTA"

    elif self.state == "ASPETTA":

```

```

self.is_moving = False
rider_rect = pygame.Rect(self.x - 25, self.y - 25, 50, 50)
if player_rect.colliderect(rider_rect):
    if self.product_type in player.items and self.qty_delivered < self.qty_needed:
        if player.transfer_cooldown == 0:
            player.items.remove(self.product_type)
            self.qty_delivered += 1
            player.transfer_cooldown = 8
    if self.qty_delivered >= self.qty_needed:
        player.money += self.reward
        self.state = "ESCI"

elif self.state == "ESCI":
    self.is_moving = True
    self.walk_tick += 1
    tx, ty = 550, SCREEN_HEIGHT + 50
    dx, dy = tx - self.x, ty - self.y
    dist = math.hypot(dx, dy)
    if dist > 4:
        self.x += (dx / dist) * self.speed
        self.y += (dy / dist) * self.speed
    else: self.active = False

def draw(self, surface):
    draw_monkey(surface, self.x, self.y, COL_DELIVEROO, False, self.walk_tick,
self.is_moving)
    pygame.draw.rect(surface, (0, 150, 135), (self.x - 12, self.y + 10, 24, 14),
border_radius=2)

    if self.state == "ASPETTA":
        pygame.draw.rect(surface, (0,0,0), (self.x - 75, self.y - 55, 150, 24), border_radius=5)
        txt = font_sm.render(f"Deliveroo: {self.qty_delivered}/{self.qty_needed}
{self.product_type}", True, COL_TEXT)
        surface.blit(txt, txt.get_rect(center=(self.x, self.y - 43)))

class UpgradeButton:
    def __init__(self, x, y, w, h, text, stat_name, cost, icon_draw_func):
        self.rect = pygame.Rect(x, y, w, h)
        self.text = text
        self.stat_name = stat_name
        self.cost = cost
        self.icon_func = icon_draw_func

    def draw(self, surface, current_money, current_val):
        is_maxed = (self.stat_name == "cap" and current_val >= 10) or (self.stat_name ==
"helper_cap" and current_val >= 6)
        if is_maxed:
            back_col = (70, 70, 70)

```



```

        txt_display = "MAX LIVELLO"
        cost_display = "---"
    else:
        can_afford = current_money >= self.cost
        back_col = COL_BTN if can_afford else (90,90,90)
        txt_display = self.text
        cost_display = f"${self.cost}"

    pygame.draw.rect(surface, back_col, self.rect, border_radius=8)
    self.icon_func(surface, self.rect.x + 12, self.rect.centery)

    txt_title = font_sm.render(txt_display, True, COL_TEXT)
    surface.blit(txt_title, (self.rect.x + 32, self.rect.y + 5))

    txt_cost = font_md.render(cost_display, True, COL_MONEY if (not is_maxed and
current_money >= self.cost) else (220,100,100))
    surface.blit(txt_cost, (self.rect.x + 32, self.rect.y + 25))

def is_clicked(self, pos, current_money, current_val):
    if self.stat_name == "cap" and current_val >= 10: return False
    if self.stat_name == "helper_cap" and current_val >= 6: return False
    if self.rect.collidepoint(pos): return current_money >= self.cost
    return False

def draw_icon_cap(surf, x, y):
    pygame.draw.rect(surf, (120,60,10), (x-6, y-6, 12, 12))
    draw_wheat_item(surf, x, y)

def draw_icon_speed(surf, x, y):
    pygame.draw.polygon(surf, COL_MONEY, [(x-6, y), (x, y-6), (x, y+6)])
    pygame.draw.polygon(surf, COL_MONEY, [(x, y), (x+6, y-6), (x+6, y+6)])

def draw_department_paths(surface):
    paths = [
        pygame.Rect(40, 110, 520, 60),
        pygame.Rect(40, 490, 520, 60),
        pygame.Rect(40, 310, 480, 60),
        pygame.Rect(350, 110, 60, 440),
        pygame.Rect(410, 300, 330, 60),
    ]
    for p in paths:
        pygame.draw.rect(surface, COL_PATH_BORDER, p)
        inner = p.inflate(-6, -6)
        pygame.draw.rect(surface, COL_PATH, inner)

# --- INIZIALIZZAZIONE ATTORI E OGGETTI ---
player = Player()

```

```

structures = [
    # --- REPARTO BANANE ---
    Structure(60, 60, 60, 60, None, 'source', 'banana', cost=0),
    Structure(150, 60, 60, 60, None, 'source', 'banana', cost=150),
    Structure(330, 60, 100, 75, COL_SHELF, 'shelf', 'banana', cost=0),
    Structure(445, 60, 100, 75, COL_SHELF, 'shelf', 'banana', cost=200),
    Structure(260, 110, 50, 50, COL_HELPER, 'helper_pod', 'banana', cost=200),

    # --- REPARTO GRANO ---
    Structure(60, 540, 60, 60, None, 'source', 'wheat', cost=250),
    Structure(150, 540, 60, 60, None, 'source', 'wheat', cost=350),
    Structure(330, 540, 100, 75, COL_SHELF, 'shelf', 'wheat', cost=0),
    Structure(445, 540, 100, 75, COL_SHELF, 'shelf', 'wheat', cost=350),
    Structure(260, 490, 50, 50, COL_HELPER, 'helper_pod', 'wheat', cost=400),

    # --- SEZIONE GALLINA / UOVA ---
    Structure(50, 295, 95, 50, None, 'processor', 'wheat', cost=500),
    Structure(260, 300, 100, 75, COL_SHELF, 'shelf', 'egg', cost=0),
    Structure(260, 395, 100, 75, COL_SHELF, 'shelf', 'egg', cost=450),
    # RISOLTO: Stand di sblocco per l'aiutante nel reparto uova posizionato vicino alla
    gallina!
    Structure(170, 300, 50, 50, COL_HELPER, 'helper_pod', 'egg', cost=350),

    # --- PIAZZA CENTRALE, CASSIERE E CESTINO ---
    Structure(620, 240, 80, 65, COL_CASH, 'cash', cost=0),
    Structure(720, 240, 50, 50, COL_CASHIER, 'helper_pod', 'cashier', cost=300),
    Structure(GAME_AREA_WIDTH - 70, 20, 50, 45, COL_TRASH, 'trash', cost=0)
]

banners = [
    DepartmentBanner(50, 175, "BANANE"),
    DepartmentBanner(50, 630, "GRANO"),
    DepartmentBanner(50, 400, "UOVA")
]

helpers_list = []
customers = []
customer_spawn_timer = 0
unlocked_product_types = ['banana']

deliveroo_rider = None
DELIVEROO_COOLDOWN = 10800
deliveroo_timer = 0

upgrades = [
    UpgradeButton(GAME_AREA_WIDTH + 10, 140, UI_PANEL_WIDTH - 20, 50, "Zaino
    Cap. +1", "cap", 50, draw_icon_cap),

```

```

    UpgradeButton(GAME_AREA_WIDTH + 10, 210, UI_PANEL_WIDTH - 20, 50, "Velocità
+10%", "spd", 60, draw_icon_speed),
    UpgradeButton(GAME_AREA_WIDTH + 10, 280, UI_PANEL_WIDTH - 20, 50, "Aiutanti
Cap. +1", "helper_cap", 100, draw_icon_cap)
]

```

```

# --- LOOP DI GIOCO ---

```

```

running = True

```

```

while running:

```

```

    clock.tick(FPS)

```

```

    cash_desk = next((s for s in structures if s.type == 'cash'), None)

```

```

    hen_station = next((s for s in structures if s.type == 'processor'), None)

```

```

    for s in structures:

```

```

        if s.type == 'cash':

```

```

            s.cashier_present = False

```

```

            s.player_present = False

```

```

    if player.transfer_cooldown > 0:

```

```

        player.transfer_cooldown -= 1

```

```

    unlocked_product_types = []

```

```

    if any(s.active for s in structures if s.type == 'source' and s.product_type == 'banana'):

```

```

        unlocked_product_types.append('banana')

```

```

    if any(s.active for s in structures if s.type == 'source' and s.product_type == 'wheat'):

```

```

        unlocked_product_types.append('wheat')

```

```

    if any(s.active for s in structures if s.type == 'processor' and s.product_type == 'wheat'):

```

```

        unlocked_product_types.append('egg')

```

```

# 1. Gestione Eventi e Click mouse

```

```

for event in pygame.event.get():

```

```

    if event.type == pygame.QUIT:

```

```

        running = False

```

```

    if event.type == pygame.MOUSEBUTTONDOWN and event.button == 1:

```

```

        m_pos = pygame.mouse.get_pos()

```

```

        for upg in upgrades:

```

```

            current_val = player.inventory_max if upg.stat_name == "cap" else
(GLOBAL_HELPER_CAP if upg.stat_name == "helper_cap" else 0)

```

```

            if upg.is_clicked(m_pos, player.money, current_val):

```

```

                player.money -= upg.cost

```

```

                if upg.stat_name == "cap":

```

```

                    player.inventory_max += 1

```

```

                    upg.cost = int(upg.cost * 1.6)

```

```

                elif upg.stat_name == "spd":

```

```

                    player.speed *= 1.10

```

```

                    upg.cost = int(upg.cost * 1.7)

```

```

                elif upg.stat_name == "helper_cap":

```

```
GLOBAL_HELPER_CAP += 1
upg.cost = int(upg.cost * 1.8)
```

```
keys = pygame.key.get_pressed()
player.move(keys)
player_rect = pygame.Rect(player.x - player.radius, player.y - player.radius,
player.radius*2, player.radius*2)
```

```
active_sources = []
active_shelves = []
current_shelves = []
```

2. Aggiornamento Mappa e Rilevamento Collisioni

```
player_colliding_with_lock = False
```

for struct in structures:

```
    struct.update()
    if struct.type == 'shelf': current_shelves.append(struct)
    if struct.active and struct.type == 'source': active_sources.append(struct)
    if struct.active and struct.type == 'shelf': active_shelves.append(struct)
```

```
if player_rect.colliderect(struct.rect):
```

```
    if not struct.active:
```

```
        if player.money >= struct.unlock_cost:
            player_colliding_with_lock = True
            if player.current_unlocking_struct == struct:
                player.unlock_timer += 1
            else:
                player.current_unlocking_struct = struct
                player.unlock_timer = 0
```

```
        if player.unlock_timer >= 180:
```

```
            player.money -= struct.unlock_cost
            struct.active = True
            if struct.type == 'helper_pod':
```

```
                helpers_list.append(Helper(struct.rect.centerx, struct.rect.bottom + 20,
struct.product_type))
```

```
            player.current_unlocking_struct = None
            player.unlock_timer = 0
```

```
        else:
```

```
            if player.current_unlocking_struct == struct:
                player.current_unlocking_struct = None
                player.unlock_timer = 0
```

```
    else:
```

```
        if player.transfer_cooldown == 0:
```

```
            if struct.type == 'source':
                if struct.stock > 0 and len(player.items) < player.inventory_max:
                    player.items.append(struct.product_type)
```

```

        struct.stock -= 1
        player.transfer_cooldown = 6

    elif struct.type == 'processor':
        if 'wheat' in player.items and struct.input_stock < struct.input_max_stock:
            player.items.remove('wheat')
            struct.input_stock += 1
            player.transfer_cooldown = 8

        if struct.stock > 0 and len(player.items) < player.inventory_max and
player.transfer_cooldown == 0:
            player.items.append('egg')
            struct.stock -= 1
            player.transfer_cooldown = 8

    elif struct.type == 'shelf':
        if len(player.items) > 0 and struct.stock < struct.max_stock:
            for i in range(len(player.items) - 1, -1, -1):
                if player.items[i] == struct.product_type:
                    player.items.pop(i)
                    struct.stock += 1
                    player.transfer_cooldown = 6
                    break

    elif struct.type == 'cash':
        struct.player_present = True
        if struct.money_pool > 0:
            player.money += struct.money_pool
            struct.money_pool = 0

    elif struct.type == 'trash':
        if len(player.items) > 0:
            player.items.pop()
            player.transfer_cooldown = 6

    if struct.active and struct.type == 'cash' and player_rect.colliderect(struct.rect):
        struct.player_present = True

if not player_colliding_with_lock:
    player.current_unlocking_struct = None
    player.unlock_timer = 0

# 3. Update Aiutanti
for helper in helpers_list:
    helper.update(active_sources, active_shelves, cash_desk, hen_station)

# 4. Logica Deliveroo
if deliveroo_rider is None:

```

```

    deliveroo_timer += 1
    if deliveroo_timer >= DELIVEROO_COOLDOWN:
        if len(unlocked_product_types) > 0:
            deliveroo_rider = DeliverooRider(unlocked_product_types)
            deliveroo_timer = 0
    else:
        deliveroo_rider.update(player_rect, player)
        if not deliveroo_rider.active: deliveroo_rider = None

# 5. Gestione Clienti e Coda Cassa
customer_spawn_timer += 1
if customer_spawn_timer > 200:
    if len(customers) < 3 + (len(unlocked_product_types) * 2) and
len(unlocked_product_types) > 0:
        customers.append(Customer(unlocked_product_types))
        customer_spawn_timer = 0

cash_queue = [c for c in customers if c.state in ("VAI_A_CASSA", "PAGA")]

for customer in customers[:]:
    crossed_out = customer.update(current_shelves, cash_desk, cash_queue)
    if crossed_out: customers.remove(customer)

# 6. Disegno Grafica di Gioco
screen.fill(COL_BG)
draw_department_paths(screen)

for banner in banners: banner.draw(screen)
for struct in structures:
    if struct.type != 'shelf': struct.draw(screen, player)

for helper in helpers_list: helper.draw(screen)
if deliveroo_rider is not None: deliveroo_rider.draw(screen)
for customer in customers: customer.draw(screen)

player.draw(screen)

for struct in structures:
    if struct.type == 'shelf': struct.draw(screen, player)

# --- INTERFACCIA UTENTE LATERALE ---
ui_rect = pygame.Rect(GAME_AREA_WIDTH, 0, UI_PANEL_WIDTH,
SCREEN_HEIGHT)
pygame.draw.rect(screen, COL_UI_BG, ui_rect)
pygame.draw.line(screen, (200,200,200), (GAME_AREA_WIDTH, 0),
(GAME_AREA_WIDTH, SCREEN_HEIGHT), 2)

center_x = GAME_AREA_WIDTH + (UI_PANEL_WIDTH // 2)

```

```

txt_logo = font_md.render("MONKEY MART", True, COL_TEXT)
screen.blit(txt_logo, txt_logo.get_rect(center=(center_x, 35)))

txt_money = font_lg.render(f"${player.money}", True, COL_MONEY)
screen.blit(txt_money, txt_money.get_rect(center=(center_x, 75)))

txt_stats = font_sm.render("POTENZIAMENTI", True, (180,180,180))
screen.blit(txt_stats, txt_stats.get_rect(center=(center_x, 115)))

upgrades[0].draw(screen, player.money, player.inventory_max)
upgrades[1].draw(screen, player.money, 0)
upgrades[2].draw(screen, player.money, GLOBAL_HELPER_CAP)

if deliveroo_rider is None:
    pct = deliveroo_timer / DELIVEROO_COOLDOWN
    pygame.draw.rect(screen, (30,30,30), (GAME_AREA_WIDTH + 10, SCREEN_HEIGHT
- 90, UI_PANEL_WIDTH - 20, 10))
    pygame.draw.rect(screen, COL_DELIVEROO, (GAME_AREA_WIDTH + 10,
SCREEN_HEIGHT - 90, int((UI_PANEL_WIDTH - 20) * pct), 10))
    txt_deliv_time = font_sm.render("Deliveroo in arrivo...", True, (160,160,160))
    screen.blit(txt_deliv_time, (GAME_AREA_WIDTH + 10, SCREEN_HEIGHT - 115))

    txt_inv = font_sm.render(f"Zaino: {len(player.items)}/{player.inventory_max}", True,
COL_TEXT)
    screen.blit(txt_inv, (GAME_AREA_WIDTH + 10, SCREEN_HEIGHT - 40))

pygame.display.flip()

pygame.quit()
sys.exit()

```